



P0-Finalen 2020

Lösungsförslag

Björnes Magasin

Av: Hugo Eberhard

Allmänna insikter och hantera indatan

Det är jobbigare att jobba med hur många dagar björnarna sover än hur många dagar de är vakna.

Datum är också jobbiga, så vi borde direkt omvandla varje datum till ett heltal mellan 0 och 364.

Björne kommer alltid vara med och vakta, så det räcker att betrakta den del av året som Björne sover, vilket blir ett intervall av dagar.

Problemet ändras inte om alla björnar somnar x dagar senare.

Testgrupp 1: $n \leq 15$

Lösning: Brute force, prova att anställa alla möjliga delmängder av björnar

Komplexitet: $O(2^n)$

Testgrupp 2: $d \leq 182$

Tolkning: Varje björn är vaken minst halva året.

Insikt: Om det går att välja några björnar som kan vakta affären tillsammans så räcker det med max två björnar utöver Björne själv. Det är för att om det finns någon dag då tre olika björnar är vakna så måste minst en av dem vara överflödig, den som vaknar "i mitten", och om det fanns fyra björnar totalt som alla var vakna minst halva året så skulle det finnas någon dag då tre av dem var vakna samtidigt, enligt lådprincipen.

Lösning: Prova alla möjligheter att anställa exakt en björn, om något av det funkar skriv ut 1. Prova annars att anställa alla björnar, om det funkar skriv ut 2. Annars skriv ut -1.

Komplexitet: $O(n)$

Testgrupp 3: $n \leq 10^5$

Insikt: Eftersom alla björnar är vakna lika länge så är det alltid bäst att som nästa björn anställa den som vaknar sist under nuvarande björns vaktpass.

Lösning 1: Det finns bara 365 dagar. Spara för varje dag om minst en björn vaknar då. Välj en björn i taget enligt insikten ovan, t.ex genom att gå igenom alla 365 dagar varje gång en ny björn ska väljas, och ta den björn som passar bäst enligt insikten.

Komplexitet: $O(n + 365^2)$

Lösning 2: Sortera alla björnar. Gå igenom i ordning, välj nya björnar enligt insikten ovan.

Komplexitet: $O(n \log n)$

Stjärnbilder

Av: Erik Amirell Eklöf

Stjärnbilder

Maximera antalet stjärnor och använd att:

min antal rymdskepp = N - max antal stjärnor.

För varje punkt i bild 1, loopa över varje punkt i bild 2.

Om dessa är samma stjärna måste $(u, v) = (x_2 - x_1, y_2 - y_1)$.

Vi vill välja u och v så att ett maximalt antal sådana antaganden blir sanna.

Räkna hur många gånger varje (u, v) förekommer (t.ex. med en map).

Max antal förekomster av samma (u, v) är max antal stjärnor.

Gruppindelning

Av: Nils Gustafsson

Förberedd av: Mattias Akke

Testgrupp 1: $n \leq 15$

Metod: Brute force. Testa alla möjliga sekvenser.

Tidskomplexitet: $O(2^n)$

Testgrupp 2: $n \leq 40$

För att sekvensen ska vara giltig kan inte platser 1 och 2 ha samma numrering. Platser 3 och 4 kan inte heller ha samma numrering. O.s.v.

Det räcker därmed att brute force:a över udda platser.

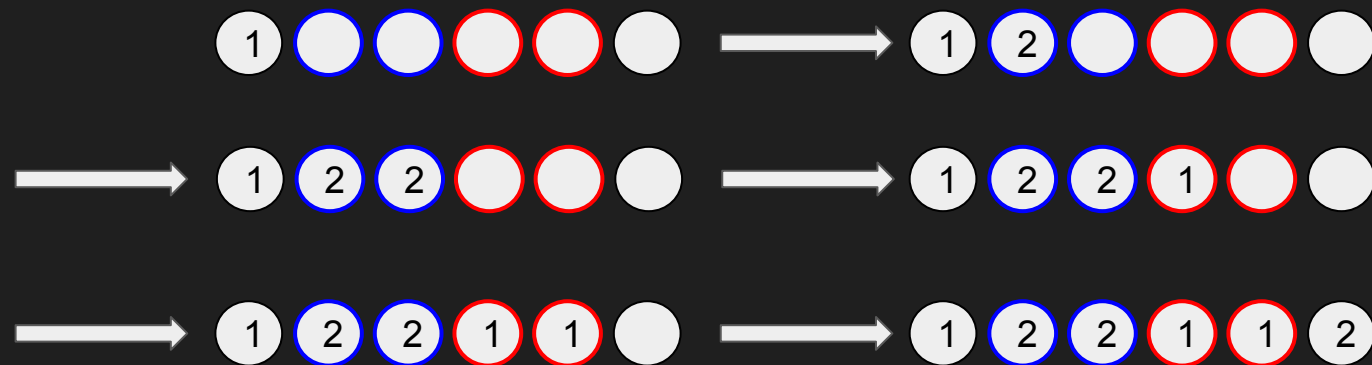
Tidskomplexitet: $O(2^{(n/2)} * m)$

Testgrupp 3: Par är alltid bredvid varandra

Metod: Girig. Välj alltid färg 1 om inte annat är framtvingat av insikten eller av att stolen i fråga ingår i ett par. Om en stol med udda index, i , ingår i ett par med stol $i+1$; skriv ut -1.

Tidskomplexitet: $O(n)$

Exempel: $n = 6$, $m = 2$: (2,3), (4,5)



Svar: 122112

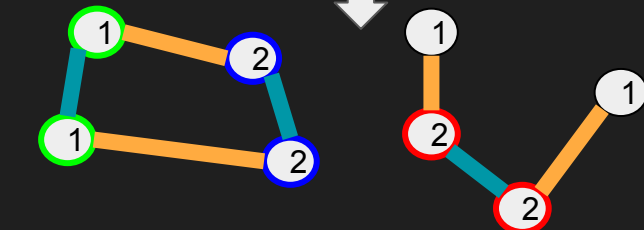
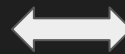
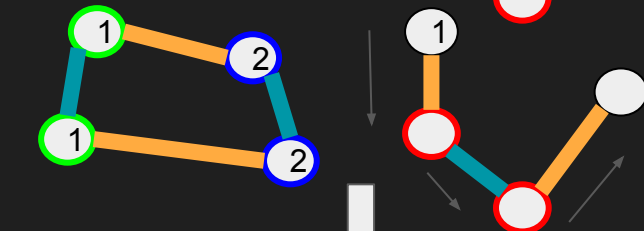
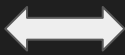
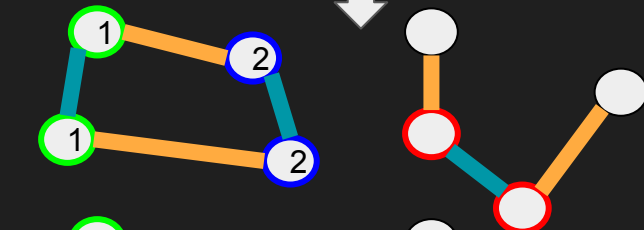
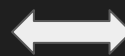
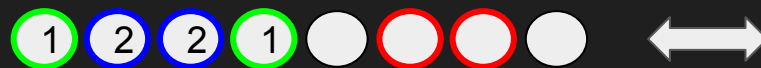
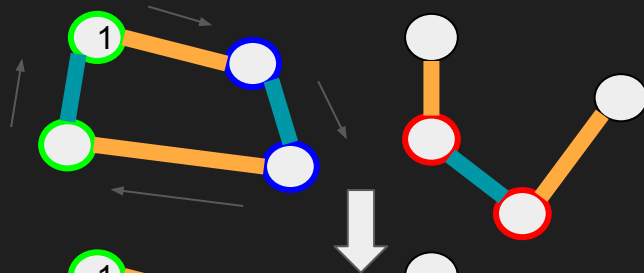
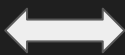
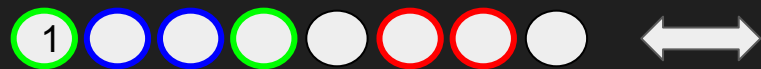
Testgrupp 4: Inga ytterligare begränsningar

Insikt: Se gruppen med stolar som en graf där paren är ihopkopplade med en “lika kant” och varje stol med udda index, i , (1-indexerat) är ihopkopplad med stol $i+1$ med en “olika kant”. Varje nod har högst grad 2, och grafen består därmed av cykler och vägar. Varje sådan komponent bestäms utifrån tilldelningen av en enda nod.

Metod: Girig. Börja från stol 1. Varje gång du kan välja grupp fritt väljer du grupp 1. Kolla igenom cykeln/vägen som noden är i och tilldela nummer på alla andra noder. (Kan ibland bli omöjligt med cykler.)

Tidskomplexitet: $O(n)$

Exempel: $n = 8$, $m = 3$: (2,3), (1,4), (6,8)



Svar: 12211221

Cellidentifikation

Av: Johan Sannemo

Förberedd av: Karl Lundstig, Simon Lindholm

Cellidentifikation

För varje delmängd av cellkomponenter är svaret “finns ej”, “vet inte” eller ett tal

Vi kan beräkna alla dessa svar i ordning av minskande delmängdsstorlek

För en viss delmängd kollar vi på om det finns celler med exakt den mängden komponenter, och på svaren för delmängderna som är ett steg större

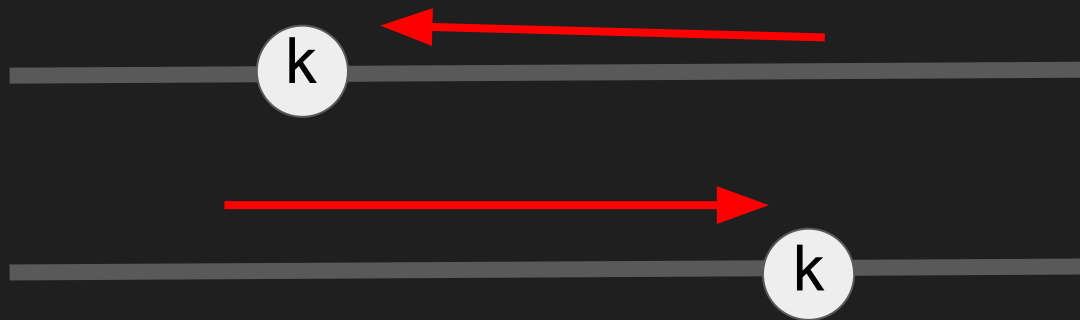
Om det bland dessa finns minst 2 typer av celler får vi “vet inte”, om bara en så får vi det talet, annars “finns ej”

Tidskomplexitet: $O(k \cdot 2^k + n + q)$

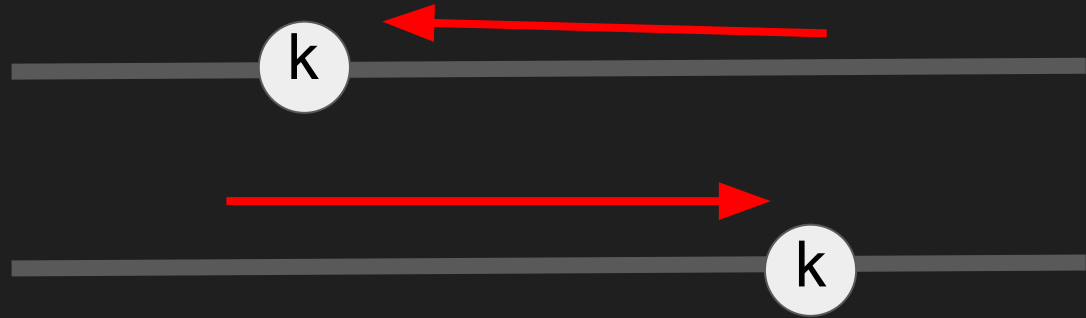
Fåglar i Träd

Av: Nils Gustafsson

Testgrupp 1:



Testgrupp 1:



Testgrupp 2: $n \leq 10$. Vi hinner testa alla $n!$ sätt för fåglarna att landa (både giltiga och ogiltiga).

Testgrupp 3: $n \leq 200$.

Insikt: Fåglarna landar i DFS-ordning.

Det finns diverse krångliga DP-lösningar i $O(n^3)$, till exempel:

Testa alla startpunkter. För en viss startpunkt p , så måste vi besöka alla noder på vägen mellan p och k , och eventuellt alla noder i några av subträden längs med vägen. Gör en subset sum DP för att hitta vilka subträd vi behöver besöka.

Testgrupp 4: $n = m$.

Insikt: Mängden noder som fåglarna landar på är sammanhängande.

=> om $n = m$ så måste k vara på ett löv, annars är svaret -1.

Gör en DFS från noden som angränsar till k , och se till så att k besöks sist.

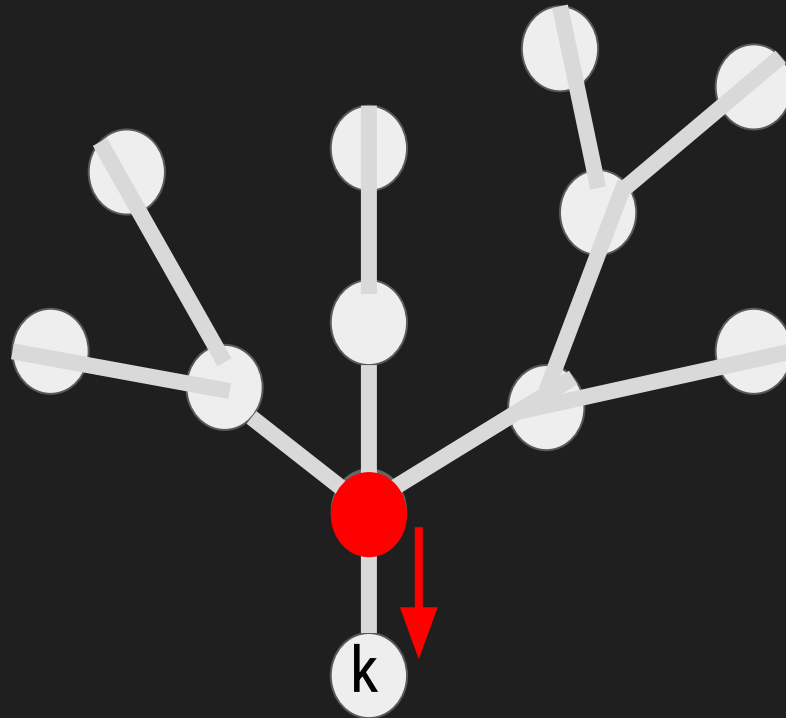
Låt ordningen fåglarna landar i vara ordningen noderna besöks i DFS:en.

Testgrupp 5: Inga ytterligare begränsningar

Om k inte har något subträd med storlek minst m , så är svaret -1 . Annars kan vi ta ett sådant subträd, och det visar sig att det alltid finns en lösning.

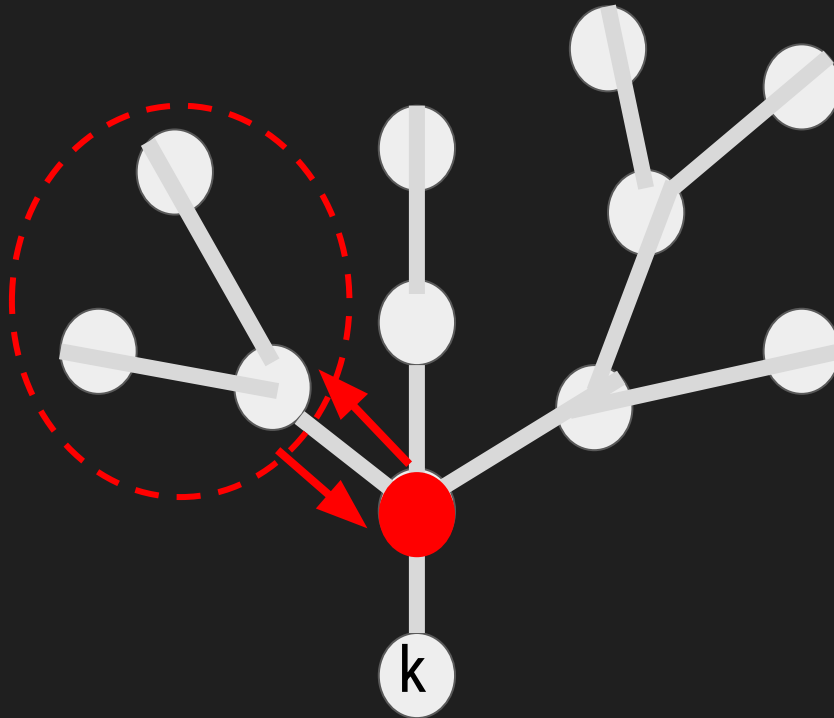
För att undvika kvadratisk tid behöver vi starta vid k och simulera processen bakifrån.

Lösning 1



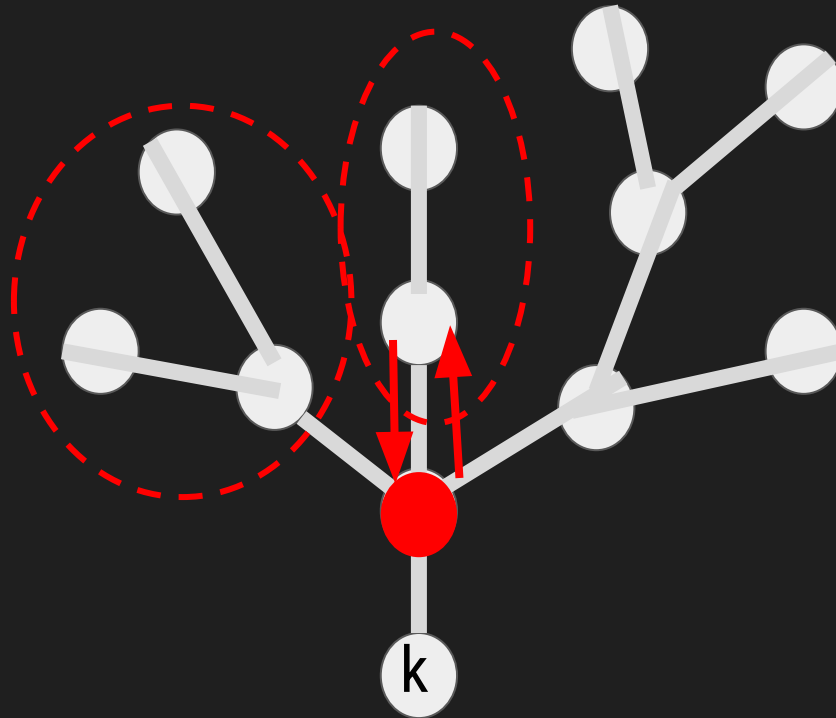
$m = 9$

Lösning 1



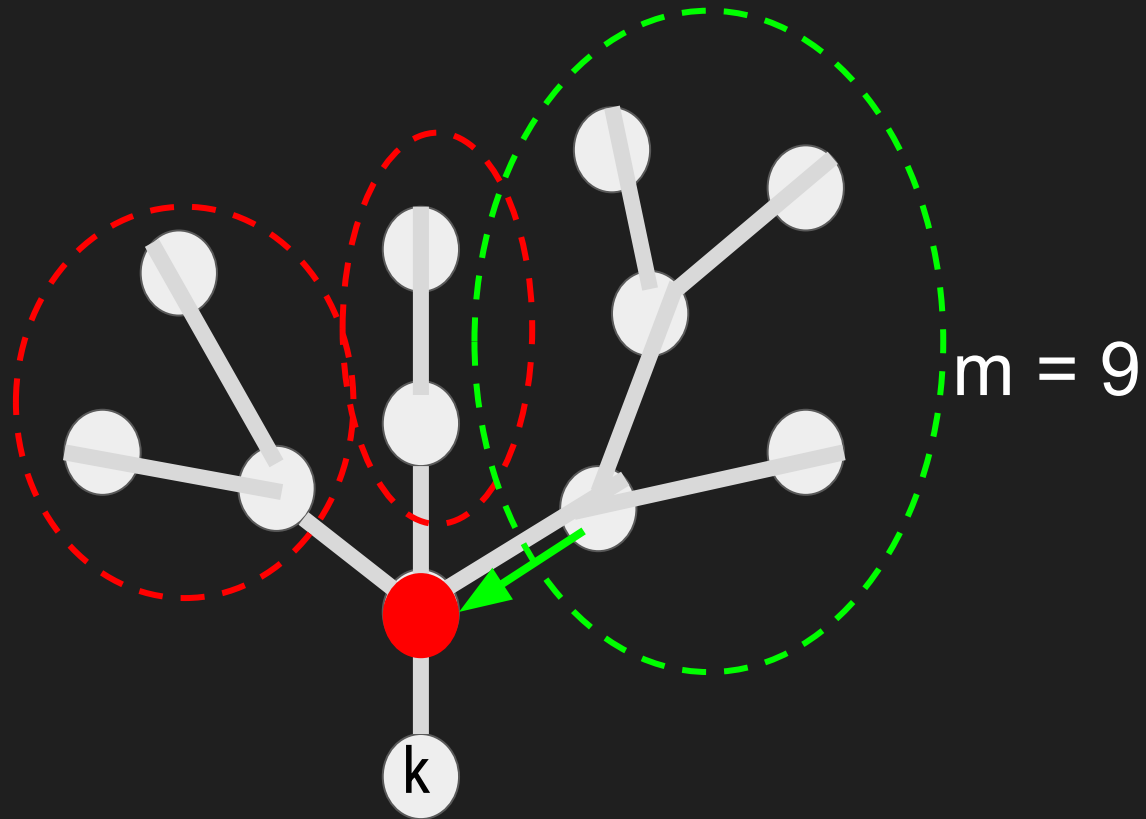
$$m = 9$$

Lösning 1

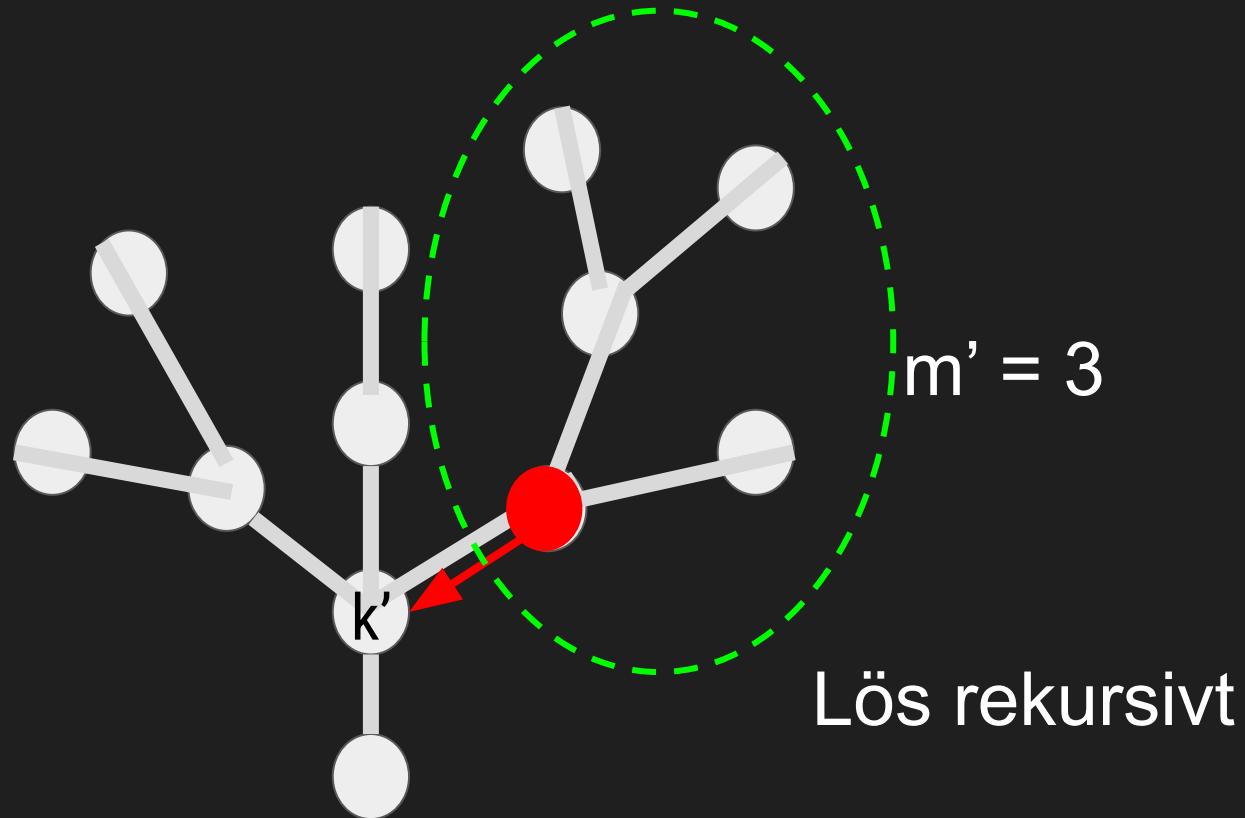


$$m = 9$$

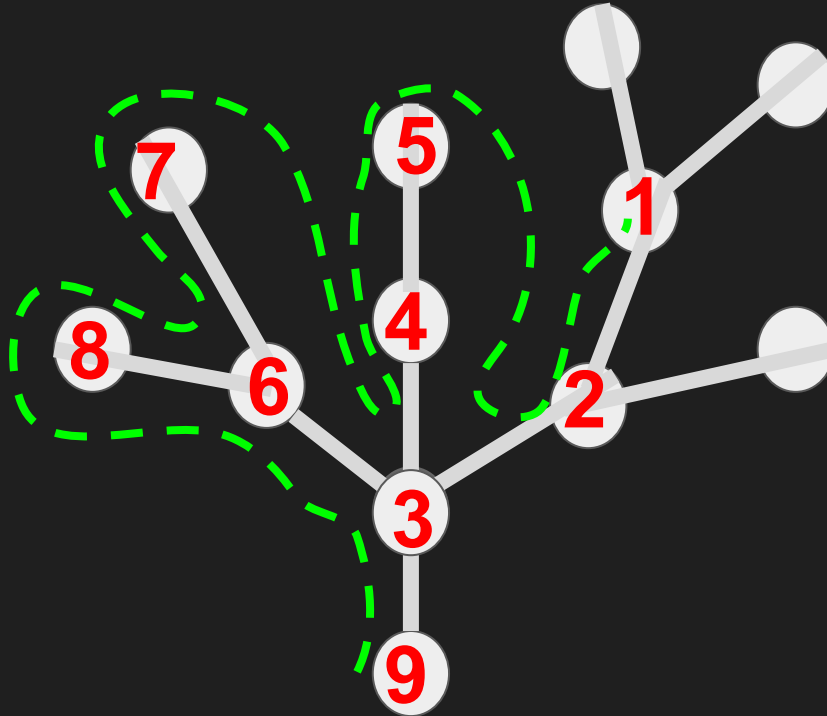
Lösning 1



Lösning 1



Lösning 2



Gör en DFS från k .

Bryt när m noder har besökts. Skriv ut noderna i omvänd post-order.

Snurriga stolpar

Av: David Wärn

Förberedd av: Hugo Eberhard

Testgrupp 1: $m \leq 5$, $n \leq 25$

Lösning: Brute force, testa all giltiga vägar. Håll koll på vilka rutor som redan är besökta, inklusive rutor där man inte svänger.

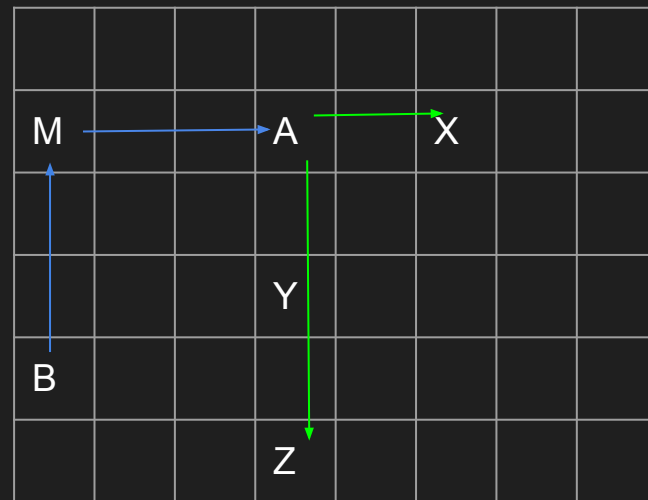
Komplexitet: $O(m \cdot 2^n)$, i praktiken blir det snabbare än så men det är lite knepigt att bevisa en bättre övre gräns

Svårare testgrupper

Insikt 1: Om vi går baklänges kommer vi svänga medsols istället för motsols.

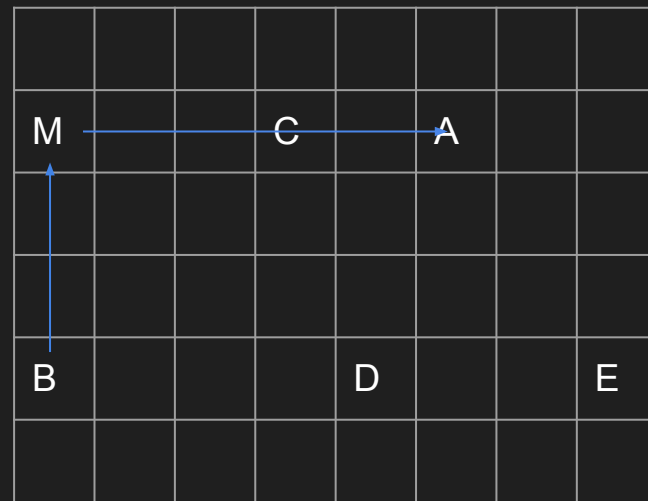
Insikt 2: Alla möjliga vägar vi kan gå kommer se ut som "dubbelspiraler".

Insikt 3: Om vi begränsar oss till att hitta "enkelspiraler" som snurrar *utåt* så är den snurriga stolpen, A, som vi befinner oss vid, och den näst senaste vi svängde vid, B, tillräcklig information för att avgöra hur vi kan fortsätta (se exempel). Vi kan antingen fortsätta rakt fram till nästa snurriga stolpe X eller svänga och fortsätta till nästa snurriga stolpe Z som ligger utanför rektangeln, med hörn i A och B.



Testgrupp 2: $m, n \leq 80$

Lösning: DP, där varje state är ett par av snurriga stolpar (A,B) samt om vi roterar medsols eller motsols, och värdet är största antalet stolpar vi kan besöka om vi håller oss innanför rektangeln med hörn i A och B och avslutar med att befinna oss i A. Vi kan beräkna varje state rekursivt på $O(n)$, eftersom vi enligt insikt 3 bara kan komma till (A,B) från states (C,B) där C ligger mellan M och A, eller states (M,D), där M är "svängpunkten" mellan A och B och D är en snurrig stolpe som vi kan ha svängt vid innan vi svängde vid B. Notera att D måste ligga innanför rektangeln med hörn i A och B, för att undvika att vi korsar vår egen väg. Vi kan alltså inte komma från (M,E) till (A,B)



Testgrupp 3: $n \leq 80$, $m \leq 10^9$

Insikt: Om det finns en x-koordinat eller y-koordinat som inte har några snurriga stolpar så är den överflödig.

Lösning: Samma som testgrupp 2, men vi använder koordinatkomprimering för att se till att m blir som mest 80.

Komplexitet: $O(n^3)$

Testgrupp 4: $n \leq 1000$, $m \leq 10^9$

Insikt 1: Om vi beräknar vår DP bottom-up, alltså att vi gör spiralerna större i varje iteration istället för att rekursera bakåt till mindre spiraler, så räcker det att fortsätta på två möjliga sätt, som specificerat i insikt 3 några slides tidigare. Då kan vi beräkna varje state på $O(1)$ istället.

Insikt 2: När vi ska kombinera enkelspiraler till dubbelspiraler så gör vi om samma beräkningar flera gånger. Vi kan även här göra en DP som för varje möjlig "gemensam punkt", varje rotationsriktning och varje "kvadrant" beräknar största möjliga enkelspiral som håller sig i den kvadranten och startar i den gemensamma punkten. Varje state beräknas på $O(1)$, eftersom vi antingen måste gå hela vägen till den gemensamma punkten i både x- och y-led, eller så räcker det att använda en mindre del av kvadranten vilket motsvarar ett annat state.

Testgrupp 4: $n \leq 1000$, $m \leq 10^9$

Lösning: Vi vill som tidigare först hitta största enkelspiral för varje par av snurriga stolpar (A,B) som håller sig inne i rektangeln med hörn i A och B, och som slutar i A. Men nu beräknar vi det iterativt, och noterar att varje spiral bara kan fortsätta på två sätt, antingen till en spiral (C,B) eller till en spiral (D,M). För att hitta C och D effektivt kan vi först ha gjort förberäkningar för att hitta nästa stolpe i varje riktning från varje punkt (vi har givetvis koordinatkomprimerat), och vi behöver även veta hur många stolpar det finns på segmentet mellan A och D vilket vi också kan förberäkna. Notera även att vi vill göra allt detta för båda rotationsriktningarna.



För att detta ska funka måste vi beräkna alla states i rätt ordning. Ett sätt att göra det är att sortera motsvarande rektanglar efter area, eller att sortera dem efter omkrets, eftersom detta ökar i varje steg.

Testgrupp 4: $n \leq 1000$, $m \leq 10^9$

När det är klart vill vi kombinera enkelspiraler till dubbelspiraler, vilket vi kan göra genom att beräkna DP:n som beskrevs i insikt 2 ovan och sen fortsätta som för testgrupp 2 och 3.

Komplexitet: $O(n^2)$

Det är ganska klurigt att få rätt på allt det här även om man har alla idéer. Till exempel så finns det för varje startposition åtta möjliga sorters spiraler som kan börja där i varje given punkt, när man ska beräkna de bästa enkelspiralerna för varje "gemensam punkt". Det är möjligt att kopiera sin kod och ändra några parametrar för varje riktning, men det blir lätt buggar som är väldigt svåra att hitta. Hur kan man göra detta effektivt, med liten risk för buggar och så att det går att hitta alla buggar som trots allt smyger sig in?